

Übungsblatt: Naive Bayes

NB.01: Wahlkampf mit Naive Bayes

a. Manuelles Training für den Datensatz aus Aufgabe DTL.01

- Prior- und bedingte Wahrscheinlichkeiten bestimmen

a)

Prior - Wahrscheinlichkeiten

Kandidat

O: Nr. 1, 2, 5, 6 $\rightarrow$ 4

M: Nr. 3, 4, 7 $\rightarrow$ 3

$\hookrightarrow P(O) = \frac{4}{7}$, $P(M) = \frac{3}{7}$

Bedingte Wahrscheinlichkeiten

Alter für O

≥ 35 : Nr. 1, 5

< 35 : Nr. 2, 6

$$\hookrightarrow P(\geq 35 | O) = \frac{2}{4}$$

$$P(< 35 | O) = \frac{2}{4}$$

Alter für M

≥ 35 : Nr. 3, 4

< 35 : Nr. 7

$$\hookrightarrow P(\geq 35 | M) = \frac{2}{3}$$

$$P(< 35 | M) = \frac{1}{3}$$

Einkommen für O

hoch : Nr. 1, 5, 6

niedrig : Nr. 2

$$\hookrightarrow P(h | O) = \frac{3}{4}$$

$$P(n | O) = \frac{1}{4}$$

Einkommen für M

hoch : Nr. 3

niedrig : Nr. 4, 7

$$\hookrightarrow P(h | M) = \frac{1}{3}$$

$$P(n | M) = \frac{2}{3}$$

Bildung für 0

Abitur : Nr. 1

Bachelor : Nr. 2, 5

Master : Nr. 6

$$\hookrightarrow P(A|0) = \frac{1}{4}$$

$$P(B|0) = \frac{2}{4}$$

$$P(Ma|0) = \frac{1}{4}$$

Bildung für 1

Abitur : Nr. 4, 7

Bachelor : Nr. 3

Master : /

$$\hookrightarrow P(A|1) = \frac{2}{3}$$

$$P(B|1) = \frac{1}{3}$$

$$P(Ma|1) = 0$$

- b) <35, niedrig, Bachelor klassifizieren

b) <35, niedrig, Bachelor

$$\text{Für } 0: \frac{4}{7} \cdot \frac{2}{4} \cdot \frac{1}{4} \cdot \frac{1}{4} = \frac{1}{28}$$

$$\text{Für } 1: \frac{3}{7} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{63}$$

Wir klassifizieren als 0

NB.02: Textklassifikation mit Naive Bayes: Spam-Erkennung

- Datensatz aufbereiten
 - Das folgende Skript in Python (auch noch separat hochgeladen) transformiert den Datensatz so, dass ham und spam durch binäre Indikatoren ersetzt werden, der Text von Sonderzeichen und Großschreibung entfernt wird und anschließend alle Wörter gezählt und vektorisiert werden. Dies geschieht ohne die häufigen Hilfsörter der Englischen Sprache wie "the" und nur wenn Wörter mindestens zweimal vorkommen, um Rauschen zu reduzieren
 - Merkmale sind hier wie häufig die entsprechenden Wörter vorkommen in Abhängigkeit davon ob es ham oder spam ist

```
import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("spam_ham_dataset.csv")

print(df.head())
print(df.columns)

df["label"] = df["label"].map({"ham": 0, "spam": 1})

# Sonderzeichen entfernen und Text kleinschreiben
def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[a-zäöüß ]", "", text)
```

```

    return text

df["text_clean"] = df["text"].apply(clean_text)

vectorizer = CountVectorizer(
    stop_words="english", # Ignoriert standardwörter wie "the"
    min_df=2              # Wörter die nur 1 mal vorkommen ignorieren
)

X = vectorizer.fit_transform(df["text_clean"])
y = df["label"]

```

- Implementierung des Klassifikators

```

# Model trainieren
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MultinomialNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

feature_names = vectorizer.get_feature_names_out()

# Top Spam und Ham
spam_log_probs = model.feature_log_prob_[1]
ham_log_probs = model.feature_log_prob_[0]

diff = spam_log_probs - ham_log_probs

```

```

top_spam = np.argsort(diff)[-10:]
top_ham = np.argsort(diff)[:10]

print("Spam:")
for i in reversed(top_spam):
    print(feature_names[i])

print("\nHam:")
for i in top_ham:
    print(feature_names[i])

```

- Wie sieht der Klassifikator aus?
 - Die Wahrscheinlichkeiten sind gespeichert in model.feature_log_prob_
 - Zeile 0 ist ham und Zeile 1 spam, wobei die Spalten jeweils Wörter sind und die Einträge dessen Häufigkeit
- Was sind die wichtigsten Begriffe?
 - Spam
 - nbsp, pills, computron, xp, width, font, cialis, href, viagra, soft
 - Ham
 - ect, enron, hpl, meter, daren, hou, ectcc, forwarded, mmbtu, xls
- Bewertung des Testergebnisses
 - Mit bis zu 97% Genauigkeit ist das Training sehr gut verlaufen und der Klassifikator sehr akkurat